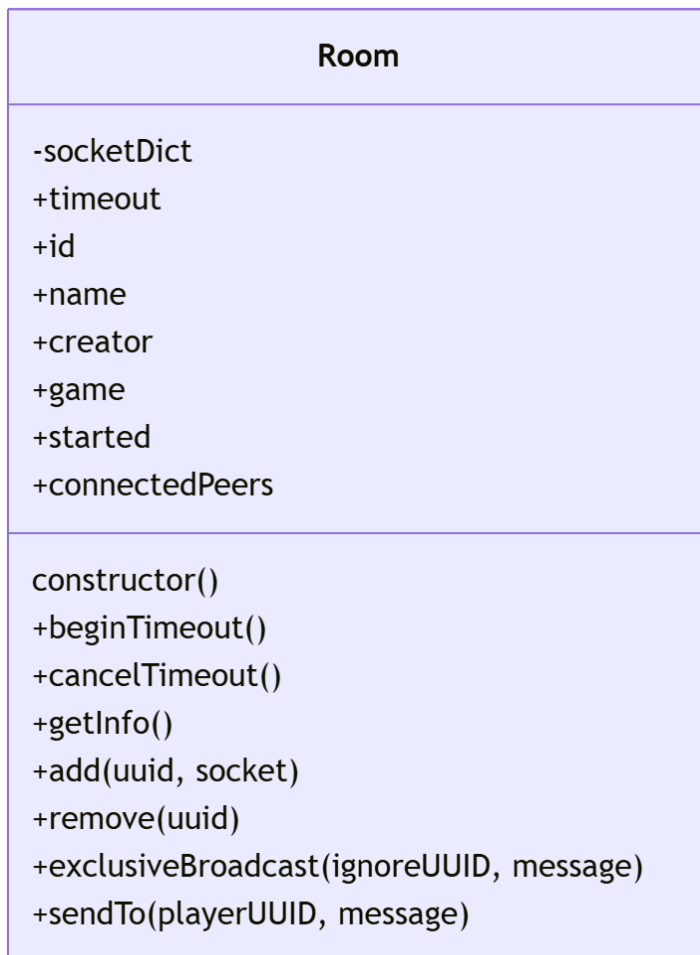


Flying Casino: UML Documentation

The following document provides some additional documentation on each of the classes. In addition to these classes, lots of functional programming was used, and can be seen in the files themselves. There are several other classes with similar names to ones presented here, those are fragments of older iterations and may be ignored. Constructors have been omitted when not explicitly defined.

From **signal_server.js**:

Room



Represents a collection of players playing the same game on the server. While most client-to-client communication is done through WebRTC, this must exist to establish the initial connection and provide peer IDs to new connections.

From **player-api.js**:

LocalPlayerData

LocalPlayerData
-name -chips -wins
constructor(name, chips, wins) +getName() +getChips() +giveChips(x) +getWins() +giveWins()

Provides an interface to store basic statistics and player information on the browser's local storage.

From **network.js**:

NetworkAPI

NetworkAPI
-socket -id -roomId -peers -pendingIceCandidates
constructor(socket, id, roomId) +getId() +getRoomId() +getPeers() +destroy() +broadcastObject(object) +broadcastStringifiedJson(json) +sendObject(object, peerId) +sendStringifiedJson(json, peerId) -send(to, json) -setupDataChannel(dataChannel) +createConnection(peerId, initiator) +setupMessageHandler()

This core class is responsible for dealing with all client-side communication using WebRTC. Its purpose is to abstract the sending and receiving of data, as well as setting up a data channel through STUN/ICE/signaling.

From **card.js**:

Card

Card
+label +value +suit
constructor(label, value, suit)

Basic functionality for all card objects.

From **deck.js**:

Deck (implements card.Card)

Deck
+handSize +Card AS ... +Card KD +Card[] cards
constructor(handSize) +reset() +shuffle() +deal(playerNumber, Card[][] hands)

Represents a deck of cards and provides the base capabilities for most card games. The (...) means that all 52 cards have their own field in this class, thus this class also functions as an enum (though it should be in a separate object).

From **poker.js**:

Poker (extends *deck.Deck*)

Poker extends Deck
constructor(handSize)

This is an abstract class that serves as the base for Poker-like games. Its only function is to change card values.

VideoPoker (extends *Poker*)

VideoPoker extends Poker
-Card[] hand
constructor() +deal() +discard(index) +getHand()

Stores game data for the game video poker.

Pot

Pot
-prize -allInLimit -currentBet -Player[] players -Pot sidePot
- constructor(Player[] players, currentBet) +bet(Player player, bet) +raise(Player player, raise) +call(Player player) +splitPot() +reward(Player[] players) +destroy()

Core class responsible for storing prizes, chips, and active bets for multiplayer games.

TexasHoldEm (extends Poker)

TexasHoldEm extends Poker
-Pot pot -Player[] players -currentBet -toPlayIndex -firstToPlayIndex -raised -round +Card[] communityCards
- constructor(Player[] players, currentBet) -sendToAllPlayers(object) +receiveAction(actionObject) +getGameState() +cardReveals() +endGame() +ante(smallBlindIndex, bet) -flop() -turn() -river() -nextTurn() +playerRaise(playerIndex, raise) +playerCall(playerIndex) +playerFold(playerIndex)

Game controller object for the game TexasHoldEm. Includes both AI actions and helpers that allow multiplayer mode.

Player

Player
+chipsRemaining +chipsBet +state +name
constructor(sendFunction)

Utility object that represents a player.

VideoPokerHTMLHandler

VideoPokerHTMLHandler
+currentChips +betInput +dealButton +hand +output
constructor(document, container) +outputString(string) +play() +discard() +endGame()

Wrapper class for updating the visuals for video poker.

TexasHoldEmHTMLHandler

TexasHoldEmHTMLHandler
+currentBet +raiseInput +raise +call +fold +hand +tableau +gameOutput +playerStates +playerSelfIndex +sendToHost
constructor(document, container, players, playerSelfIndex, sendToHost) +outputString(string) +updatePlayerState(playerIndex, state) +updatePlayerBalance(playerIndex, chipsBet, chipsRemaining) +renderHand(hand) +receiveAction(actionObject)

Wrapper class for interactions and visuals for TexasHoldEm

From **blackjack.js**:

BlackJackHTMLHandler (implements *deck.Deck* and *card.Card*)

BlackjackHTMLHandler
+Deck deck +Card[] playerHand +Card[] dealerHand +HTMLElement dealerCardsEl +HTMLElement dealerValueEl +HTMLElement playerCardsEl +HTMLElement playerValueEl
+getHand(hand) +hit(hand) +stay() +dealerPlay() +renderCards(container, hand, hideSecond) +updateValue(el, hand, hidden)

Wrapper class providing visuals for the game blackjack. Looks slightly different due to it being implemented in the same file as the game, but it functions the same as the other HTMLHandlers.

From **holdem.js**:

PokerAI (extends *poker.Player*)

PokerAI extends Player
+aiIndex +_act +_hand +_comm
constructor(aiIndex) +setActionFn(fn) +_receive(msg) +_decide()

Object that is used in place of a player when an AI is playing. The AI is mainly confined to singleplayer.